```
# Program to calculate calories and other information for
# recipes.
import sys
import os
import platform
from loguru import logger


import config_mycal


from PySide6.QtCore import QSize, Qt ,QCoreApplication
from PySide6.QtGui import QAction
from PySide6.QtWidgets import QVBoxLayout , QGridLayout
from PySide6.QtWidgets import QWidget
from PySide6.QtSql import QSqlDatabase , QSql,QSqlTableModel




from PySide6.QtWidgets import (
QApplication,
    QCheckBox,
    QComboBox,
    QDateEdit,
    QDateTimeEdit,
    QDial,
    QDoubleSpinBox,
    QFileDialog,
    QFontComboBox,
    QLabel,
    QLCDNumber,
    QLineEdit,
    QMainWindow,
    QProgressBar,
    QPushButton,
    QRadioButton,
    QSlider,
    QSpinBox,
    QTimeEdit,
    QTableView,
    QMenuBar,
)


class MainWindow(QMainWindow):
    def __init__(self, Title =  None , pos_x = 100 , pos_y = 500, size_x = 800 , size_y = 600):
        super().__init__()  # this will the allow to use this as a Subclass

        #with self and using QMainWindow, we can now acces everything through self
        # first we give it the Title


        # Read in the configuration
        self.SetupConfig()


```

```
58          # some constants for json file
59
60
61          # the logging system
62          self.SetupLogger()
63          self.log_level = self.CM.log_level
64
65
66          # first get the password
67  #        self.GetPwd()
68
69          # connect to database
70          self.ConnectDataBase()
71
72          if Title != None:
73              self.setWindowTitle(Title)
74          else:
75              self.setWindowTitle("QMainWindow")
76
77
78          # now we set up geometry
79          self.SetSize(size_x,size_y)
80          self.SetPosition(pos_x , pos_y)
81
82
83          self.AddMyMenu()
84
85          # now do the layout
86          self.MyLayout()
87
88      def MyLayout(self):
89          '''Here I do the layout of the window'''
90
91          # here we create a button widget in the mddle of the mainwindow.
92          mylayout = QGridLayout()
93          mylayout.addWidget(QDoubleSpinBox(),0,1)
94          mylayout.addWidget(QComboBox(),0,0)
95          mylayout.addWidget(QDial(),1,0)
96          mylayout.addWidget(QDateTimeEdit(),1,1)
97
98          # connect to widget
99          mywidget = QWidget()
100         mywidget.setLayout(mylayout)
101         self.setCentralWidget(mywidget)
102
103
104     def SetPosition(self,pos_x,pos_y):
105         self.move(pos_x,pos_y)
106
107     def SetSize(self,size_x,size_y):
108         #self.setBaseSize(size_x,size_y) only works on QWidget
109         self.resize(size_x,size_y)
110
111
112
113     def AddMyMenu(self):
114         '''Now we add a menu'''
```

```
115         #menubar = QMenuBar(None)
116         menubar = self.menuBar()
117 
118         file_menu = menubar.addMenu("File")
119         db_menu = menubar.addMenu("Database")
120 
121         # database handling:
122          #No menu will show without an action defined
123         #Also Quit does not show up since this is already in the intrinsic Python menu.
124         # Same for exit
125 
126     # here come a few actions, which will be called from the button qwidget
127     # When you press a button, it will trigger one or everal events through slots
128     # one thing which is important to note ist that one click on a button can have many different meanings
129     # DON"T FORGET TO PUT, self IN QACTION
130 
131         #here we define the actions: File open section
132         file_action = QAction("Open", self)
133         file_action.setStatusTip("Opens a file")
134         file_action.triggered.connect(self.OpenFile)
135         file_menu.addAction(file_action)
136 
137 
138         #file_menu.addAction("Quit")
139 
140 
141 
142 
143         #saving the file
144         save_action = QAction("Save", self)
145         save_action.setShortcut("CMD+S")
146         save_action.setStatusTip('save File')
147         save_action.triggered.connect(self.SaveFile)
148         file_menu.addAction(save_action)
149 
150         db_action_1 = QAction("List Table",self)
151         db_action_1.setStatusTip('List database tables')
152         db_action_1.triggered.connect(self.ShowTables)
153         db_menu.addAction(db_action_1)
154         db_action_2 = QAction("save",self)
155         db_action_2.setStatusTip('List database tables')
156         db_action_2.triggered.connect(self.ShowTables)
157         db_menu.addAction(db_action_2)
158 
159 
160         file_menu.addAction("MyExit")
161         file_menu.addAction("None")
162 
163 
164 
165 
166     def ConnectDataBase(self):
167         ''' establish contact to database'''
168 
169 
170         #instantiate the connection
171         self.mycal_db  = QSqlDatabase.addDatabase(self.CM.db_system)
```

```
172         self.mycal_db.setHostName("localhost")
173         self.mycal_db.setDatabaseName(self.CM.db_name)
174         self.mycal_db.setUserName(self.CM.db_user)
175         self.mycal_db.setPassword(self.GetPwd())
176 
177 
178 #        self.mycal_db.setPassword(self.CM.pwd)
179 
180         result = self.mycal_db.open()
181         if(result):
182             logger.info('connection succsessful')
183             # here we get the connection name
184             # will be needed when we want to close the connection
185             self.connection_name = self.mycal_db.connectionName()
186             logger.info(' database connection name %s' % self.connection_name)
187             #self.ShowTables()
188         else:
189             logger.error('connection failed, exciting')
190             sys.exit(0)
191 
192     def GetPwd(self):
193         temp = self.CM.cryptofile
194         if os.path.exists(temp):
195             with open(temp, 'r') as file:
196                 self.password = password = file.read().rstrip()
197                 return password
198 
199 
200         else:
201             logger.error('crytopfile not found, exciting')
202             sys.exit(0)
203 
204 
205 
206     def OpenFile(self):
207         ''' Open file dialog'''
208 
209         self.FileName = QFileDialog.getOpenFileName(self)
210 
211     def SaveFile(self):
212         name = QFileDialog.getSaveFileName(self,'Save File')
213         myfile = open(name,'w')
214         text = self.textEdit.toPlainText()
215         myfile.write(text)
216         myfile.close()
217 
218 
219 
220 
221 
222     def SetupConfig(self):
223         mysystem = platform.system()
224 
225         if mysystem == 'Darwin':
226             conf_dir = '/Users/klein/git/qt_exercises/config/'
227         elif mysystem == 'Linux':
228             conf_dir = '/home/klein/git/qt_exercises/config/'
```

```
229         else:
230             print(' This os is not supported %s' % mysystem)
231             sys.exit(0)
232         config_file = conf_dir + 'config_mycal.json'
233 
234         self.CM = CM = config_mycal.MyConfig(config_file)
235         self.log_level = CM.log_level
236         self.log_output = CM.log_output
237 
238 
239 
240 
241 
242 
243     def SetupLogger(self):
244 
245 
246         logger.remove(0)
247         #now we add color to the terminal output
248         logger.add(sys.stdout,
249                 colorize = True,format="<green>{time}</green>  {function}  {line}  {level}   <level>{message}</level>" ,
250                 level = self.log_level)
251 
252 
253 
254         fmt =  "{time} - {name}-  {function} -{line}- {level}  - {message}"
255         logger.add(self.log_output, format = fmt , level = self.log_level,rotation="1 day")
256 
257 
258         # set the colors of the different levels
259         logger.level("INFO",color ='<black>')
260         logger.level("WARNING",color='<green>')
261         logger.level("ERROR",color='<red>')
262         logger.level("DEBUG",color = '<blue>')
263 
264         return
265 
266     def ShowTables(self):
267         '''prints all the tables in the databe'''
268         self.table_list = self.mycal_db.tables(type=QSql.Tables)
269         for k in range(0,len(self.table_list)):
270             print(self.table_list[k])
271 
272         #Create list box
273         self.mybox = QComboBox()
274         #Insert at index 0 the whole list
275         self.mybox.insertItems(0,self.table_list)
276         # create independent window
277 
278         self.setCentralWidget(self.mybox)
279 
280 
281         return
282 
283 
284 
285 
```

```
286     def ViewTable(self,table):
287 
288         model = QSqlTableModel(db = self.mycal_db)
289         model.setTable(table)
290         model.select()
291 
292         table = QTableView()
293         table.setModel(model)
294         #self.setCentralWidget(table)
295 
296 
297 app = QApplication(sys.argv)
298 window = MainWindow(Title = "GridLayout")
299 #window.SetSize(800,500)
300 #window.SetPosition(100,500)
301 
302 window.setStyleSheet("background-color: white;")
303 #window.CreateCalendar()
304 
305 
306 
307 window.show()
308 window.ConnectDataBase()
309 window.ShowTables()
310 window.ViewTable('Recipes')
311 # now run the app
312 app.exec()
```